Nee confusion mainly **“DevOps lo actual ga enni tools untayi, avi workflow lo ekkada vastayi?”** ane point lo undi.

Real DevOps Tool Flow

Oka typical production project ni example ga teesukunte:
Developer
↓
Git / GitHub
↓
Jenkins
↓
SonarQube
↓
Build
↓
Test
↓
Docker
↓
Docker Image
↓
ECR / Docker Registry
↓
GitOps Repo
↓
Argo CD
↓
Kubernetes / EKS
↓
Helm / Kustomize
↓
Application Pods
↓
Prometheus
↓
Grafana
Parallel ga infrastructure side:
Terraform
↓
AWS Infrastructure
↓
VPC
EKS
IAM
EC2
ECR
Configuration-management/automation side:
Ansible
↓
Servers / Configuration / Application setup
Legacy Java application unte:
Tomcat
↓
Java Application
So **nee list lo unna prati tool ki different purpose undi.**

---

1. Git / GitHub
First concept.
Developer code maintain cheyyadaniki:
Developer
↓
Git
↓
GitHub
Git is version control.
Example:
main
dev
feature/login
feature/payment

Code changes, branches, commits, pull requests etc. ikkada manage chestaru.

---

2. Jenkins
Git nunchi code vachaka automation start cheyyadaniki Jenkins use cheyyachu.
GitHub
↓
Jenkins
↓
Pipeline
Jenkins:
Build
Test
Package
Docker Build
Push Image
lanti tasks automate chestundi.
So Jenkins ni mainly:
CI/CD automation server
ga think cheyyachu.

---

3. SonarQube
Idi **code quality / static analysis** side.
Jenkins pipeline lo integrate cheyyachu.
Example:
GitHub
↓
Jenkins
↓
SonarQube
↓
Build
↓
Test
SonarQube check cheyyagalige things:
Bugs
Code Smells
Vulnerabilities
Duplications
Code Quality
Example:
Developer bad code push chesadu anuko.
Jenkins:
Build
↓
SonarQube Scan
↓
Quality Gate
Quality gate fail ayithe pipeline stop cheyyachu.
So:
Jenkins
→ Pipeline orchestration

SonarQube
→ Code quality analysis

---

4. Maven / Gradle
Nee list lo lekapoyina **Java projects lo very important**.
Suppose Java application:
Java Code
↓
Maven
↓
Build
↓
.jar
or:
Gradle
↓
Build
So actual Java DevOps flow ila undachu:
GitHub

↓
Jenkins
↓
SonarQube
↓
Maven
↓
Tests
↓
.jar

---

5. Tomcat
Tomcat kuda context batti vastundi.
Tomcat is a **Java application server / servlet container**.
Traditional Java application architecture:
Developer
↓
Git
↓
Jenkins
↓
Maven
↓
WAR
↓
Tomcat
↓
Application
Example:
myapp.war
↓
Tomcat
↓
Java Application
But modern microservices world lo every application Tomcat meeda deploy avvalsina requirement ledu.
Spring Boot applications often:
Spring Boot
↓
Embedded Tomcat
↓
Docker
↓
Kubernetes
So **Tomcat ni mandatory DevOps tool ani anukovaddu**.
It's mainly relevant when the application architecture uses Java/Tomcat.

---

6. Docker
Ikkada application ni containerize chestam.
Application
↓
Dockerfile
↓
Docker Build
↓
Docker Image
Example:
myapp:v10
Then registry:
Docker Image
↓
ECR

---

7. Terraform
Terraform **application deployment tool kaadu**.
Idi infrastructure create/manage cheyyadaniki.
Example AWS:
Terraform
↓
VPC
↓
Subnets

↓
IAM
↓
EKS
↓
Node Groups
↓
ECR
So Terraform ni separate branch laga think cheyyi.
Terraform
↓
AWS
↓
Infrastructure
Application deployment:
Git
↓
Jenkins
↓
Docker
↓
ECR
↓
Argo CD
↓
EKS
Terraform EKS ni create cheyyachu.
Argo CD EKS lo applications deploy/manage cheyyachu.
Very important distinction:
Terraform
→ Infrastructure

Argo CD
→ Kubernetes Application

---

8. Ansible
Ansible mainly **configuration management and automation**.
Example:
100 Linux servers unnayi.
Manual ga:
Install Java
Install Nginx
Create users
Copy configuration
Restart service
cheyyadam badulu:
Ansible
↓
100 Servers
automation cheyyachu.
Example:
Ansible
↓
Install Java
↓
Configure application
↓
Start service
Traditional infrastructure environments lo Ansible chala common.
Kubernetes environment lo kuda use cases undachu, but Kubernetes application deployment kosam Argo CD unte Ansible ni same purpose ki necessarily use cheyyaru.
Simple:
Ansible
→ Server configuration / automation

Terraform
→ Infrastructure provisioning

Argo CD
→ Kubernetes GitOps deployment

---

9. Helm
Kubernetes applications complex ayyaka Helm useful.
Raw YAML:
deployment.yaml
service.yaml
configmap.yaml
ingress.yaml
chala applications ki maintain cheyyadam difficult.
Helm:
Helm Chart
↓
Templates
↓
Values
↓
Kubernetes Manifests
Example:
values.yaml

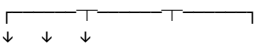
```
image:
 repository: myapp
 tag: v10

replicas: 3
```

Argo CD Helm ni manage cheyyagaladu.
So:
Git
↓
Helm Chart
↓
Argo CD
↓
Kubernetes

---

10. Kubernetes
Ikkada actual application containers run avtayi.
Docker Image
↓
Kubernetes
↓
Deployment
↓
Pods
Example:
Deployment
↓



Pod Pod Pod
Kubernetes:

- Containers run chestundi
- Scaling manage chestundi
- Self-healing provide chestundi
- Service discovery
- Rolling updates
- Scheduling

etc.

---

11. Argo CD
Kubernetes deployments ni GitOps way lo manage cheyyadaniki.
GitOps Repo
↓
Argo CD
↓
Kubernetes / EKS
Argo CD:
Git Desired State
↓
Compare

↓
Cluster Actual State
↓
Reconcile
Example:
Git = v10
Cluster = v9

→ OutOfSync
Auto-sync/self-healing configured unte desired state ki reconcile cheyyachu.

---

12. Prometheus
Application Kubernetes lo run avtundi.
Ippudu:
“Application/cluster performance ela undi?”
ani telusukovali.
Prometheus:
Kubernetes
↓
Metrics
↓
Prometheus
Metrics example:
CPU
Memory
Request Count
Error Rate
Latency
Pod Metrics
Node Metrics
Prometheus mainly **metrics collection and monitoring** kosam.

---

13. Grafana
Prometheus data ni visualize cheyyadaniki Grafana commonly use chestaru.
Application / Kubernetes
↓
Prometheus
↓
Grafana
↓
Dashboard
Example dashboard:
CPU Usage 72%
Memory Usage 65%
Requests 10k/min
Error Rate 1.2%
Pods 10/10
So:
Prometheus
→ Collect/store/query metrics

Grafana
→ Visualize metrics

---

14. Logging Tools
Nee list lo lekapoyina production lo logging kuda important.
Common:
ELK
Elastic
Logstash
Kibana
or:
Loki
Promtail / Fluent Bit
Grafana
Flow:
Application
↓
Logs
↓
Logging System

↓
Search / Dashboard
Example:
Pod CrashLoopBackOff
↓
Check Logs
↓
Database connection refused

---

15. Security Tools
Production DevOps lo security kuda untundi.
Example:
SonarQube
→ Code Quality / Static Analysis

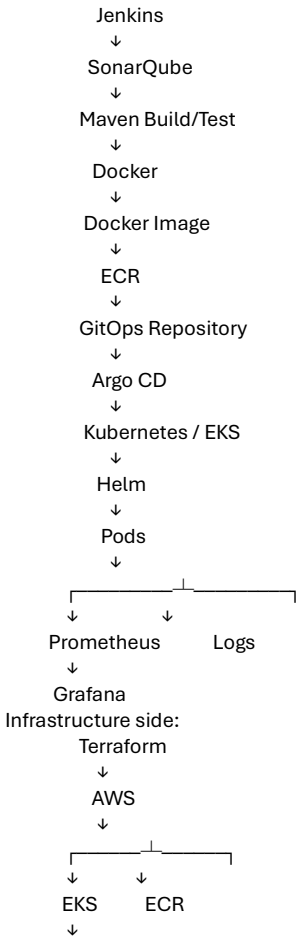
Trivy
→ Container/Image vulnerability scanning

Secret Scanning
→ Credentials detection
Pipeline:
Git
↓
Jenkins
↓
SonarQube
↓
Build
↓
Docker
↓
Trivy
↓
Registry
↓
Deployment
Exact order organization batti change avvachu.

So all tools ni categories lo remember cheyyi
Idi **most important table** bro.

Category	Tools	Main Work
Version Control	Git, GitHub	Code management
CI/CD	Jenkins	Pipeline automation
Code Quality	SonarQube	Code analysis
Build	Maven, Gradle	Application build
App Server	Tomcat	Java application runtime
Containerization	Docker	Container/image
Registry	ECR, Docker Hub	Image storage
Infrastructure	Terraform	Infrastructure provisioning
Configuration	Ansible	Server/config automation
Orchestration	Kubernetes	Container orchestration
Packaging	Helm	Kubernetes application packaging
GitOps CD	Argo CD	Kubernetes deployment/reconciliation
Monitoring	Prometheus	Metrics
Visualization	Grafana	Dashboards
Logging	ELK, Loki	Logs
Security	Trivy, SonarQube etc.	Security/quality

One Real-Time Project Example
Suppose company ki **Java Spring Boot application** undi.
App name:
Payment Service
Full flow:
Developer
↓
Git / GitHub
↓



Infrastructure side:

If traditional Java deployment:

GitHub
↓
Jenkins
↓
Maven
↓
WAR
↓
Tomcat

But containerized modern deployment:

GitHub
↓
Jenkins
↓
Maven
↓
Docker
↓
ECR
↓
Kubernetes / EKS

Tomcat may be **inside the container** if the application needs it, rather than being separately installed on a VM.

So Nuvvu Learning Order Ila Pettuko

Random ga:

❌

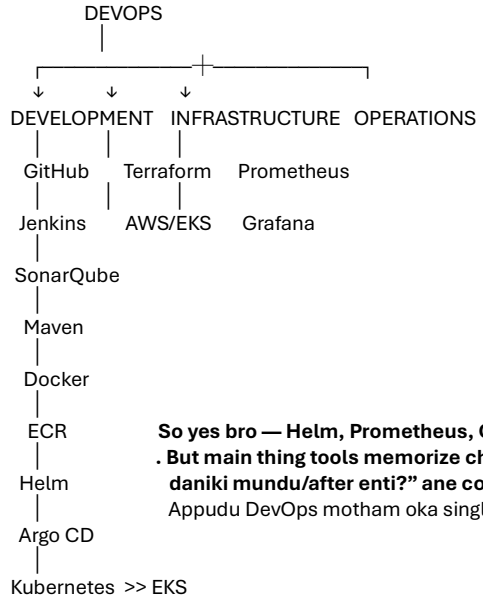
Jenkins
Terraform
Grafana
Tomcat
Ansible
Argo CD
Helm

ila start cheyyaku.
Instead **workflow + purpose** order lo:

1. Linux
↓
2. Networking Basics
↓
3. Git + GitHub
↓
4. Maven / Gradle basics
↓
5. Jenkins
↓
6. SonarQube
↓
7. Docker
↓
8. Docker Registry / ECR
↓
9. Kubernetes
↓
10. Kubernetes YAML
↓
11. Helm
↓
12. AWS
↓
13. EKS
↓
14. Terraform
↓
15. GitOps
↓
16. Argo CD
↓
17. Prometheus
↓
18. Grafana
↓
19. Logging
↓
20. Ansible
↓
21. Security tools
↓
22. Advanced Kubernetes

Tomcat మాత్రం separate application-technology topic. Java/Tomcat project unte nerchuko; every DevOps engineer ki Tomcat deep ga mandatory kaadu.

One final mental picture



So yes bro — Helm, Prometheus, Grafana, Terraform, Ansible, Tomcat, SonarQube ila chala tools untayi. But main thing tools memorize cheyyadam kaadu; “ee problem kosam ee tool, workflow lo ekkada daniki mundu/after enti?” ane connection ardham chesukoval ,
Appudu DevOps motham oka single story laga kanipistundi